

13/9/2025

Task-9.1

Implement Exceptions and exceptional handling in python.

Aims To implement exception and exceptional handling in python.

Algorithm

- 1) Start the program
- 2) Initializes a list of grades
- 3) prompts the user to enter the index of the grade
- 4) Attempts to display the grade at the index
- 5) If the index is out of range, catches the index error.

Program

```
# initialize the list of grades
grades = [85, 90, 78, 92, 88]

# Display the grades list
print("Gades list:", grades)

# prompt the user to enter the index of the grade
they want to view
Index = int(input("Enter the index to view:"))

# Attempt to display the grade
print("The grade at index", Index, "is", grades[Index])

except index error:
    # Handle the case where the index is out of range
    print("Invalid error.")

# Handle the case where the input is not an integer
try:
    Index = int(input("Enter the index to view:"))
except ValueError:
    print("Invalid error")
```


(Part 1: Input - output) Input - output

Output:

Grades list : [85, 90, 78, 92, 88]

Enter the index of the grade you want to view : 10

Invalid error : please enter a valid index.

Storage list that contains. Translated program with output
and input set has been successfully executed and the output

NAME - CODE	NO
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10

Output:

Enter the numerator = 10

Enter the denominator = 0

Error!

Error : Division by zero is not allowed.

Task - 9.2

Aim: To develop a python calculator program that performs basic arithmetic operations.

Algorithm:

- 1) start the program
- 2) prompts the user to enter two numbers
- 3) Attempts to divide the numerator by the denominator
- 4) If the denominator is zero, catches the Zero Division Error and displays an error message.

Program:

```
# function to perform division
```

```
def divide_numbers():
```

```
    try:
```

```
        # prompt the user to enter
```

```
        numerator = float
```

```
        # prompt the user to enter
```

```
        denominator = float
```

```
        # Attempt to perform division
```

```
        result = numerator / denominator
```

```
        print("result is [result]")
```

```
    except ZeroDivisionError:
```

```
        # Handle division by zero error.
```

```
        print("Error")
```

```
    except ValueError:
```

```
        # Handle invalid input
```

```
        print("Error")
```

```
# call the function to execute.
```

```
divide_numbers()
```


To develop a Python calculator program that
performs basic arithmetic operations

Requirements:

- 1. Start the program
- 2. Prompt the user to enter two numbers
- 3. Prompt the user to enter the operator (+, -, *, /)
- 4. Perform the calculation and display the result
- 5. Handle division by zero error

Program:

def main():
 # Function to perform division

def divide(a, b):
 try:
 return a / b

Output:

Enter a number: 15

Exception occurred = Invalid Age

Result:

Task-9.3

Aim: To building a python application to determine if a person is eligible to vote based on their age.

Algorithm:

- 1) Define the custom Exception
- 2) prompt the user for input
- 3) check if the age is below 18.
- 4) Raise an exception.
- 5) Handle the exception with a custom.

Program:

```
# define python user-defined exceptions
class Invalid Age Exceptions
    "Raised when the input value
# you need to guess this number
number = 18
```

Try:

```
Input_num = int(input())
```

```
if input_num < number:
```

```
    raise Invalid Age Exception
```

```
else:
```

```
    print("Eligible to vote")
```

```
except Invalid Age Exception
```

```
    print("Exception occurred")
```

Result: Thus the program for implement exceptions and Exceptional handling is Executed and verified Successfully

VEL TECH	
EX NO.	
PERFORMANCE (5)	
RESULT AND ANALYSIS (5)	
VIVA VOCE (5)	
RECORD (5)	
TOTAL (20)	
SIGN WITH DATE	